

ЛАБОРАТОРНА РОБОТА № 1

Тема: Розгортання інфраструктури розробки: Miniconda, VS Code, Git/GitHub та базові режими виконання Python

Мета роботи: Опанувати повний цикл розгортання та конфігурації професійного інженерного середовища для розробки програмного забезпечення мовою Python. Набути практичних навичок створення та ізоляції віртуальних середовищ за допомогою пакетного менеджера Miniconda, інтеграції середовища розробки Visual Studio Code із підсистемою налагодження debugpy та мовним сервером PyLance, налаштування розподіленої системи контролю версій Git та віддаленого репозиторію GitHub. Дослідити фундаментальні відмінності виконання програмного коду в інтерактивній консолі REPL, термінальному пакетному режимі та інтерактивному блокноті Jupyter Notebook.

Студента III курсу KI-24-1 групи

Ступінь вищої освіти бакалавр
(бакалавр, магістр)

Спеціальність 123 – «Комп'ютерна інженерія»

Освітня програма Комп'ютерна інженерія

Яцентюка Євгенія Олександровича

1. Мета роботи

Опанувати повний життєвий цикл розгортання та конфігурації інженерного робочого середовища для розробки програмного забезпечення мовою Python. Набути практичних навичок створення й ізоляції віртуальних середовищ за допомогою пакетного менеджера Miniconda, інтеграції середовища розробки Visual Studio Code з підсистемою налагодження debugpy та мовним сервером PyLance, налаштування розподіленої системи контролю версій Git. Дослідити механізми виконання програмного коду в термінальному пакетному режимі та інтерактивному блокноті Jupyter Notebook, а також здійснити низькорівневу інспекцію параметрів рантайму CPython.

2. Постановка індивідуального завдання (Варіант 2)

1. Створити та налаштувати конфігураційні файли проєкту (environment.yml, .gitignore, launch.json).
2. Провести інспекцію версій середовища виконання CPython за допомогою спеціалізованих модулів і системних змінних: sys.version_info, platform.python_build(), sys.hexversion.
3. Реалізувати алгоритм математичного побітового аналізу 32-бітного числа sys.hexversion для визначення мікроверсії (patch level) інтерпретатора та верифікувати її збіг із кортежем sys.version_info.
4. Дослідити просторові накладні витрати пам'яті для вбудованих скалярних і складених структур даних за допомогою функції sys.getsizeof.
5. Перевірити функціонування внутрішнього пулу малих цілих чисел (Small Integer Cache) для меж інтервалу [-5, 256] та об'єктів поза ним.
6. Оформити результати у вигляді консольного скрипта system_diagnostics.py, інтерактивного блокнота exploration_notebook.ipynb та зафіксувати історію змін засобами Git

3. Хід виконання роботи

3. Хід виконання роботи

3.1. Створення ізолюваного середовища Conda та файлова структура

Для ізоляції системних бібліотек підготовлено специфікацію залежностей у файлі `environment.yml`

```
name: ce_lab_env
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.10
  - ipykernel
  - pip
```

Для запобігання потраплянню службових та тимчасових файлів до репозиторію налаштовано файл `.gitignore`, у якому заблоковано директорії кешу CPython (`__pycache__`), папки віртуальних середовищ, системні налаштування IDE та тимчасові чекпоїнти блокнотів (`.ipynb_checkpoints`)

```

(ce_lab_env) C:\Users\Janis>conda info --envs

# conda environments:
#
# * -> active
# + -> frozen
base                C:\Users\Janis\miniconda3
b-env               C:\Users\Janis\miniconda3\envs\b-env
ce_lab_env          * C:\Users\Janis\miniconda3\envs\ce_lab_env


(ce_lab_env) C:\Users\Janis>conda list python
# packages in environment at C:\Users\Janis\miniconda3\envs\ce_lab_env:
#
# Name                                Version           Build                Channel
_python_abi3_support                 1.0               hd8ed1ab_3           conda-forge
brotli-python                        1.2.0             py310h967a849_3      conda-forge
cpython                             3.10.22           py310hd8ed1ab_0      conda-forge
ipython                              8.30.0            py310haa95532_0      conda-forge
python                               3.10.21           hb00fc5c_0            conda-forge
python-dateutil                      2.9.0post0        py310haa95532_2      conda-forge
python-fastjsonschema                2.22.2            pyhcf101f3_0         conda-forge
python-gil                           3.10.22           hd8ed1ab_0            conda-forge
python-json-logger                   4.2.0             pyhd8ed1ab_0          conda-forge
python-tzdata                        2026.3            pyhd8ed1ab_0          conda-forge
python_abi                           3.10              2_cp310              conda-forge

```

Рис. 1 — Верифікація створення та активації ізолюваного середовища `ce_lab_env` засобами **Conda**

3.2. Конфігурація відлагоджувача VS Code (*launch.json*)

Для виконання покрокового трасування коду створено конфігураційний файл `launch.json`, що взаємодіє з підсистемою `debugpy` через інтегрований термінал:

```

{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python: Current File",
      "type": "debugpy",
      "request": "launch",
      "program": "${file}",
      "console": "integratedTerminal"
    }
  ]
}

```

3.3. Розробка модуля діагностики *system_diagnostics.py*

У розробленому скрипті функція `inspect_cpython_microversion()` забезпечує виконання індивідуального завдання Варіанта 2.

Числове значення `sys.hexversion` розбивається за 32-бітною схемою кодування:

- **Байт 1 (біти 24–31):** Major version
- **Байт 2 (біти 16–23):** Minor version
- **Байт 3 (біти 8–15):** Micro version
- **Байт 4 (старші 4 біти, 4–7):** Release level (0xA — alpha, 0xB — beta, 0xC — candidate, 0xF — final)
- **Байт 4 (молодші 4 біти, 0–3):** Serial number

Для вилучення мікроверсії виконано бітовий зсув на 8 розрядів вправо з подальшим накладанням порозрядної бітової маски 0xFF:

```
decoded_micro = (sys.hexversion >> 8) & 0xFF
```

Лістинг реалізації індивідуального завдання (Варіант 2):

```
def inspect_cpython_microversion() -> dict:
    v_info = sys.version_info
    build_no, build_date = platform.python_build()
    hex_val = sys.hexversion

    # Побітове декодування sys.hexversion
    major_hex = (hex_val >> 24) & 0xFF
    minor_hex = (hex_val >> 16) & 0xFF
    micro_hex = (hex_val >> 8) & 0xFF
    release_level_code = (hex_val >> 4) & 0x0F
    serial = hex_val & 0x0F

    level_map = {0xA: "alpha", 0xB: "beta", 0xC: "candidate", 0xF: "final"}

    return {
        "major": v_info.major,
        "minor": v_info.minor,
        "micro": v_info.micro,
        "releaselevel": v_info.releaselevel,
        "serial": v_info.serial,
        "build_number": build_no,
        "build_date": build_date,
```

```
    "raw_hex": f"0x{hex_val:08X}",
    "decoded_micro": micro_hex,
    "release_level_str": level_map.get(release_level_code,
f"0x{release_level_code:X}"),
    "is_consistent": (v_info.micro == micro_hex),
}
```

3.4. Результати консольного тестування

Запуск скрипта здійснено командою `python system_diagnostics.py`. Результати діагностики середовища:

```
=====
                        ЗВІТ СИСТЕМНОЇ ДІАГНОСТИКИ СЕРЕДОВИЩА PYTHON
=====
Операційна система:      Windows 10 (64bit)
Апаратний процесор:      AMD64 Family 23 Model 96 Stepping 1, AuthenticAMD
Версія інтерпретатора:   3.10.21 (CPython)
Компілятор ядра CPython:  MSC v.1942 64 bit (AMD64)
Порядок байтів системи:   LITTLE-ENDIAN
Активоване Conda-env:     ce_lab_env
Шлях до інтерпретатора:  C:\Users\Janis\miniconda3\envs\ce_lab_env\python.exe
Ліміт глибини рекурсії:   1000 кадрів
Квант перемикання GIL:    0.005 c
-----
                        ВАРІАНТ 2: ІНСПЕКЦІЯ ВЕРСІЙ ТА МІКРОВЕРСІЙ CPYTHON
-----
Структура sys.version_info:  Major=3, Minor=10, Micro=21
Рівень релізу / Serial:      final (серійний №: 0)
Збірка platform.python_build: Ter/Хеш='main', Дата='Aug 27 2026 14:35:39'
Шістнадцятковий sys.hexversion: 0x030A15F0
Мікроверсія з hexversion:    21 (Рівень: final)
Верифікація мікроверсії:     УСПІШНО (Дані узгоджені)
-----
                        ПРОСТОРОВІ НАКЛАДНІ ВИТРАТИ СТРУКТУР ДАНИХ (CPython 64-bit)
-----
| Тип даних та стан          | Розмір у пам`яті (Байти) |
-----
| int (0)                    | 24                        |
| int (2^64)                 | 36                        |
| float                      | 24                        |
| complex                    | 32                        |
| bool                       | 28                        |
| NoneType                   | 16                        |
| str (empty)                | 49                        |
| str (ASCII, len=20)        | 69                        |
| list (empty)               | 56                        |
| tuple (empty)              | 40                        |
| dict (empty)               | 64                        |
| set (empty)                | 216                      |
-----
                        ВЕРИФІКАЦІЯ МЕХАНІЗМУ SMALL INTEGER CACHE
-----
| Число   | Адреса id(a)   | Адреса id(b)   | a is b   | Статус пулу |
-----
| -5      | 2677875277872 | 2677875277872 | True     | КЕШОВАНЕ    |
| 0        | 2677875278032 | 2677875278032 | True     | КЕШОВАНЕ    |
| 100      | 2677875281232 | 2677875281232 | True     | КЕШОВАНЕ    |
| 256      | 2677875286224 | 2677875286224 | True     | КЕШОВАНЕ    |
| 257      | 2677881543792 | 2677881543824 | False    | ДИНАМІЧНЕ   |
| 1000     | 2677881543664 | 2677881543792 | False    | ДИНАМІЧНЕ   |
=====
Час повної діагностики середовища: 84.5832 мс
```

3.5. Робота з системою контролю версій Git

Репозиторій ініціалізовано локально за допомогою команди `git init`. Після перевірки стану невідстежуваних файлів (`git status`) здійснено індексацію робочої області (`git add .`) та зафіксовано перший комміт з описом згідно зі специфікацією Conventional Commits

```
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\new\labs\lab1> git init
Initialized empty Git repository in C:/Users/Janis/Desktop/ypok/Програмування на Python. Базовий рівень\new\labs\lab1/.git/
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\new\labs\lab1> git status

On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        environment.yml
        exploration_notebook.ipynb
        launch.json
        system_diagnostics.py

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\new\labs\lab1> git add .
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\new\labs\lab1> git commit -m "feat(infra): ініціалізація проекту, налаштування conda env, VS Code та діагностичного модуля"
[master (root-commit) d008480] feat(infra): ініціалізація проекту, налаштування conda env, VS Code та діагностичного модуля
5 files changed, 199 insertions(+)
create mode 100644 .gitignore
create mode 100644 environment.yml
create mode 100644 exploration_notebook.ipynb
create mode 100644 launch.json
create mode 100644 system_diagnostics.py
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\new\labs\lab1> git log --oneline --graph
* d008480 (HEAD -> master) feat(infra): ініціалізація проекту, налаштування conda env, VS Code та діагностичного модуля
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\new\labs\lab1>
```

4. Посилання на віддалений репозиторій

Профіль: <https://github.com/kefir4ikk>

Репозиторій лабораторної роботи :

https://github.com/kefir4ikk/progPython_YevgeniiYatsentiuk/tree/master/labs/lab1

Контрольні питання:

1. У чому полягає системна відмінність між ізоляцією робочого середовища засобами стандартного модуля `venv` та дистрибутива Miniconda при роботі з низькорівневими C-бібліотеками комп'ютерної інженерії?

Модуль `venv` ізолює лише Python-пакети й використовує системний інтерпретатор та бібліотеки ОС. Miniconda ізолює також сам інтерпретатор, бінарні C/C++-бібліотеки та системні утиліти, що важливо для пакетів із нативними залежностями.

2. Яким чином влаштована внутрішня модель пам'яті еталонного інтерпретатора CPython? Чому скалярний нуль типу `int` займає 28 байтів у 64-розрядній системі, тоді як у мові C тип `int` потребує лише 4 байти?

У CPython кожне число — повноцінний об'єкт: він містить лічильник посилянь, покажчик на тип та поле розміру, а значення зберігається в масиві «цифр». Службові поля у 64-бітній системі дають 24 байти (28 в інших версіях CPython), тоді як `int` у C — лише значення розміром 4 байти.

3. Опишіть трирівневу архітектуру збереження станів файлів у системі контролю версій Git (Working Tree, Staging Area, Local Repository). Яке математичне перетворення забезпечує цілісність графа комітів DAG?

Це пул заздалегідь створених об'єктів цілих чисел, які повторно використовуються замість створення нових. Діапазон обрано емпірично: такі числа найчастіше трапляються в програмах (індекси, лічильники, коди символів), а розширення пулу збільшило б споживання пам'яті без помітного виграшу.

4. Чим принципово відрізняється виконання коду в інтерактивній консолі REPL від запуску скрипта через термінал та виконання у клієнт-серверному блокноті Jupyter Notebook?

Є три області: Working Tree (робочі файли), Staging Area (індекс) та Local Repository (історія комітів). Цілісність графа DAG забезпечує хешування SHA-1: хеш коміту залежить від дерева файлів і хеша батька, тому зміна будь-якого елемента історії змінює всі наступні хеші.

5. Яку роль відіграє конфігураційний файл `.vscode/launch.json` та підсистема `debugpy` під час покрокового трасування програми за точками зупинки (breakpoints)?

REPL виконує окремі інструкції в інтерактивній сесії; скрипт запускається цілком у новому процесі інтерпретатора; Jupyter виконує комірки в ядрі IPython, яке зберігає стан між комірками й взаємодіє з клієнтом через ZeroMQ.